

Highlights

LoopFuzz: Closed-Loop Runtime Admission Control for LLM-Guided Stateful Protocol Fuzzing

Ketang Chen, Xiaolei Ren

- LoopFuzz validates LLM-generated protocol-fuzzing interventions at runtime.
- Runtime admission checks parseability, acceptability, reachability, and gain.
- Counterexamples guide local repair of failed protocol-message hypotheses.
- State-aware scheduling targets underexplored inferred protocol states.
- Archived summaries compare LoopFuzz with AFLNet, ChatAFL, NS-Fuzz, and MBFuzzer.

LoopFuzz: Closed-Loop Runtime Admission Control for LLM-Guided Stateful Protocol Fuzzing

Ketang Chen^a, Xiaolei Ren^{a,*}

^a*Macau University of Science and Technology, Macau, China*

Abstract

Stateful protocol fuzzing is difficult because syntactically valid messages may still violate the live session state. This problem is amplified when large language models (LLMs) are inserted directly into fuzzing campaigns. We present LoopFuzz, a closed-loop runtime admission-control design for LLM-guided stateful protocol fuzzing. LoopFuzz treats model outputs as bounded hypotheses. It derives message hypotheses from RFC fragments, seed traces, and observed responses, validates candidate interventions through parseability, response-grounded acceptability, state reachability, and downstream gain, and uses counterexamples for local refinement. It also couples admission control with state-aware scheduling so that model assistance is invoked under adaptive plateau conditions and directed toward underexplored IPSM frontier states. We re-audit the archived summaries using the terminal `b_abs` and `edges` fields. On nine off-the-shelf targets, the LoopFuzz configuration (`chatafl_opt`) obtains the highest mean `b_abs` on six targets and the highest mean `edges` on all nine targets against AFLNet and ChatAFL. Sig-

*Corresponding author: Xiaolei Ren. Email address: `xlren@must.edu.mo`

Email addresses: `3250006913@student.must.edu.mo` (Ketang Chen),
`xlren@must.edu.mo` (Xiaolei Ren)

nificant `b_abs` wins versus both primary baselines appear on Pure-FTPd, Live555, and Lighttpd1; significant `edges` wins appear on bftpd, ProFTPD, Pure-FTPd, and Live555. Auxiliary comparisons remain mixed: among eight NSFuzz targets, NSFuzz has the highest `b_abs` on five, while `chatafl_opt` has the highest reported `edges` on seven. The MQTT comparison against MBFuzzer is reported separately because the available `chatafl_opt` MQTT runs are much shorter and MBFuzzer reports `branch_covered` rather than `b_abs`.

Keywords: Protocol fuzzing, Large language models, Runtime admission control, Stateful testing, Counterexample-guided refinement

1. Introduction

Network protocol implementations remain difficult security targets because correctness depends on interaction history rather than isolated inputs Pham et al. (2020); Natella (2022); Ba et al. (2022a); Hu et al. (2024); Zhang et al. (2024a). An FTP server, for example, rejects a well-formed `PASS` command if no valid `USER` command has established the login state. Unlike file fuzzing, protocol fuzzing must preserve message syntax, temporal order, and request-response progress across a multi-step session Zhang et al. (2024a); Song et al. (2025); Luo et al. (2023); Wu et al. (2024); Redini et al. (2021); Dong et al. (2025).

Traditional coverage-guided fuzzers work well for stateless parsers. They fit network servers poorly when later behavior depends on earlier valid exchanges Böhme et al. (2017); Herrera et al. (2021); Wu et al. (2022); Zhu and Böhme (2021); Ba et al. (2022b). Stateful greybox fuzzers such as AFLNet

15 and StateAFL partially close this gap. They infer progress from responses
16 and replay valid traces Pham et al. (2020); Natella (2022); Ba et al. (2022a);
17 Natella and Pham (2021); Andronidis and Cadar (2022); Luo et al. (2023);
18 Hu et al. (2024). Even so, byte-level mutation still struggles to synthesize
19 the context-sensitive messages needed to cross deep protocol boundaries.

20 Large language models offer a plausible source of such messages. ChatAFL
21 and later LLM-assisted fuzzers show that a model can extract structure from
22 specifications, logs, or documentation and produce fluent candidates Meng
23 et al. (2024); Xia et al. (2024); Wang et al. (2024); Ma et al. (2024); Wang
24 et al. (2023); Lyu et al. (2024); Deng et al. (2023, 2024). But fluency is not
25 enough. A request can be grammatically plausible and still be invalid for the
26 live session state. Such executions spend budget without improving queue
27 quality.

28 LoopFuzz places a runtime admission controller between LLM generation
29 and queue admission. It treats the LLM as a hypothesis generator rather
30 than an oracle. It derives message hypotheses from available RFC fragments,
31 seed traces, and observed responses. It stores failed executions as counterex-
32 amples and uses them for counterexample-guided refinement (CEGR) rather
33 than unconstrained regeneration. It also uses inferred protocol state ma-
34 chine (IPSM) feedback to prioritize frontier states and rare transitions, and
35 to trigger assistance only when ordinary mutation stalls. The name *LoopFuzz*
36 emphasizes closed-loop admission and refinement, avoiding any suggestion of
37 formal verification of protocol semantics, model behavior, or server correct-
38 ness.

39 The contributions of this paper are as follows:

- 40 • **Hypothesis-Driven Message Modeling.** LoopFuzz derives message
41 grammars and field constraints from RFC fragments, seed traces, and
42 observed responses, and maintains them as explicit runtime hypotheses.
- 43 • **A Closed-Loop Runtime Admission Boundary.** LoopFuzz inserts
44 a runtime admission controller between the LLM and the queue. The
45 controller checks parseability, response-grounded acceptability, state
46 reachability, and downstream gain before admission.
- 47 • **Counterexample-Guided Local Refinement.** LoopFuzz preserves
48 failed request-response traces as counterexamples and uses them to
49 patch one field constraint or production at a time, which constrains
50 model freedom and stabilizes refinement.
- 51 • **State-Aware Scheduling.** LoopFuzz leverages IPSM topology, state
52 productivity, and adaptive plateau detection to prioritize underexplored
53 frontier states and to request model assistance only when mutation
54 alone stalls.
- 55 • **An Empirical Study of Runtime-Validated Assistance.** We
56 evaluate LoopFuzz on a repeated-run benchmark archive. The study
57 includes nine off-the-shelf primary comparisons against AFLNet and
58 ChatAFL using `b_abs` and `edges`, auxiliary four-way NSFuzz compar-
59 isons on eight targets, a separately qualified MBFuzzer/`chatafl_opt`
60 Mosquitto comparison, and policy ablations of the evaluated controller.

61 Accordingly, the paper evaluates LoopFuzz as a campaign-control pol-
62 icy, not as a standalone message generator. In stateful protocol fuzzing,

generation matters only when it survives runtime evidence and improves downstream queue quality.

2. Background and Motivation

Table 1: LoopFuzz is the only listed system that combines LLM hypotheses with runtime admission and online refinement; representative prior fuzzers differ in input prior, state signal, and admission boundary.

System	Input prior	State signal	Admission rule	Online refinement
Coverage-guided mutation fuzzing B'ohme et al. (2017); Herrera et al. (2021)	Byte mutation	None	None	No
AFLNet Pham et al. (2020)	PCAP mutation	Response	Novelty	No
StateAFL Natella (2022)	PCAP mutation	Memory	Novelty	No
ChatAFL Meng et al. (2024)	LLM requests	History text	Direct admission	No
LoopFuzz (ours)	LLM hypotheses	IPSM frontier + validation	Runtime admission	Yes

Table 1 contrasts LoopFuzz with representative fuzzers. Prior LLM-assisted protocol fuzzers improve input fluency, but they leave admission largely open-loop. We study a narrower question: does closed-loop runtime admission control make LLM proposals more useful in a long-running stateful campaign?

2.1. Coverage-Guided Fuzzing vs. Network Targets

A coverage-guided fuzzer operates through a repeated feedback cycle of seed selection, mutation, execution, coverage measurement, and novelty-based queue update Herrera et al. (2021); Wu et al. (2022); Zhu and B'ohme (2021); Ba et al. (2022b). This loop is effective for file parsers, but network targets impose three constraints that matter here:

1. **Temporal state dependence:** server behavior depends on the order and semantics of earlier messages, not only on the current pay-

79 load Pham et al. (2020); Natella (2022); Ba et al. (2022a); Hu et al.
80 (2024).

81 **2. Strict message format:** protocols require delimiters, field layouts,
82 and often checksum or length consistency, so many mutated messages
83 die before they reach deeper logic Srivastava and Payer (2021); Fu et al.
84 (2022); Hu et al. (2024).

85 **3. Execution instability:** sockets, forking daemons, and asynchronous
86 response handling complicate both coverage collection and campaign
87 throughput Andronidis and Cadar (2022).

88 2.2. Stateful Greybox Fuzzing (SGF)

89 SGF addresses this gap by treating the target as an *inferred protocol state*
90 *machine* (IPSM). Formally, the IPSM can be viewed as a Mealy machine
91 $M = (Q, I, O, \delta, \lambda, q_0)$, where:

- 92 • Q is the set of inferred protocol states;
- 93 • I is the set of client messages;
- 94 • O is the set of observed server responses;
- 95 • δ maps a state and input to a successor state;
- 96 • λ maps a state and input to an observed response; and
- 97 • q_0 is the initial connection state.

98 AFLNet-style fuzzers infer Q from observed responses and use that struc-
99 ture to bias later exploration toward rare or promising parts of the protocol.
100 Here, the IPSM matters for one reason. It gives LoopFuzz a notion of progress
101 that is richer than raw edge coverage alone.

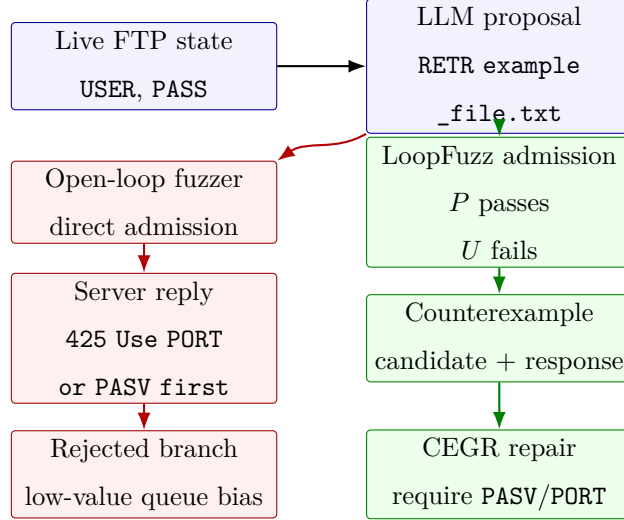


Figure 1: Semantic drift occurs when the model proposes `RETR` before the session opens a data channel; open-loop fuzzing admits the rejected branch, whereas LoopFuzz rejects the candidate, records the `425` counterexample, and repairs the missing `PASV/PORT` precondition.

2.3. LLM Integration into Protocol Fuzzing

Large language models make it easier to propose structure-aware protocol messages from RFC text, interaction history, or observed responses. That helps with syntax and message diversity, but it does not solve the harder control problem: a fluent request can still be wrong for the live session state.

2.4. A Motivating Example: Semantic Drift

Consider an FTP fuzzing campaign against ProFTPD, as illustrated in Fig. 1. The fuzzer executes `USER` and `PASS`. It then queries the LLM for the next request, and the model returns `RETR example_file.txt`. The request is syntactically fluent but semantically premature. The client lacks an active data channel via `PASV` or `PORT`. The server rejects it with `425 Use PORT or`

113 **PASV first.**

114 An open-loop LLM fuzzer admits this **RETR** request because it looks flu-
115 ent. This is *semantic drift*. The request is locally plausible, but it violates the
116 protocol precondition for the current session state. The fuzzer then mutates
117 a rejected branch instead of crossing the missing state transition. LoopFuzz
118 blocks this drift. The runtime admission controller treats the **425** reply as
119 negative evidence. It rejects the candidate and records the failed trace as a
120 counterexample. CEGR then repairs the missing state precondition instead
121 of regenerating the full continuation. This example captures the paper’s cen-
122 tral claim: in stateful protocol fuzzing, the value of LLM assistance depends
123 on closed-loop runtime admission, not on surface fluency. We use semantic
124 drift as an operational motivation for rejection and CEGR repair, not as a
125 separately estimated global rate.

126 **3. Related Work**

127 We position LoopFuzz against the prior lines that are closest in mecha-
128 nism and evaluation.

129 Earlier structure-aware fuzzing used explicit message structure, inferred
130 snippets, token constraints, or grammar-level guidance to preserve valid-
131 ity Feng et al. (2021); Salls et al. (2021); Srivastava and Payer (2021); Fu
132 et al. (2022); Bulekov et al. (2023); Fioraldi et al. (2021); Myung et al. (2022).
133 These systems improve input quality. They do not address the control prob-
134 lem that arises when an external model proposes state-dependent protocol
135 messages.

136 Stateful greybox and state-aware network-service fuzzing address protocol

137 interaction more directly. AFLNet, StateAFL, NSFuzz, and the broader SGF
138 line infer or expose state structure from server responses, process memory,
139 or program variables Pham et al. (2020); Natella (2022); Qin et al. (2023);
140 Ba et al. (2022a); Natella and Pham (2021). They then use that structure
141 to prioritize rare or productive traces.

142 Later systems extend this idea to packet-sequence exploration, format-
143 sensitive protocol fuzzing, and communication-heavy environments Luo et al.
144 (2023); Hu et al. (2024); Wu et al. (2024); Zhang et al. (2024b,a); Song
145 et al. (2025); Kim et al. (2022); Fiterau-Brostean et al. (2022); Sagonas and
146 Typaldos (2023); Zhang et al. (2025). LoopFuzz builds on this line, but adds a
147 runtime-validated admission boundary for LLM interventions. SGF, NSFuzz,
148 Bleem, Logos, and MBFuzzer are therefore important external comparison
149 points beyond AFLNet and ChatAFL. The present evaluation archive does
150 not include direct head-to-head reproduction against that broader set, so we
151 treat that omission as a limitation rather than as evidence that those systems
152 are weaker.

153 Adjacent work studies state-aware or interaction-heavy fuzzing in robotic
154 policies, firmware, Linux drivers, data-distribution middleware, hardware,
155 and embedded systems Kim et al. (2021); Aafer et al. (2021); Scharnowski
156 et al. (2022); Zhao et al. (2022); Ryu et al. (2024); Trippel et al. (2022);
157 Beckmann and Steffan (2023). These systems are not protocol fuzzers in the
158 narrow sense. They still support the broader point that state feedback and
159 interaction context can dominate raw mutation quality.

160 Target-aware scheduling and semantic validation form another relevant
161 line. Directed greybox fuzzing studies how to steer search toward high-value

162 targets Böhme et al. (2017); Du et al. (2022); Lin et al. (2025). Work on
163 seed selection and strategy adaptation studies how to spend budget more
164 effectively Herrera et al. (2021); Wu et al. (2022); Zhu and Böhme (2021);
165 Ba et al. (2022b). Semantic validation and fuzz-driver generation show that
166 stronger inputs often require an explicit acceptance interface Chen et al.
167 (2021); Zhang et al. (2021). LoopFuzz inherits that lesson and applies it to
168 LLM proposals.

169 Related work also studies how to infer dependencies or exploit guidance
170 from data-flow and learned signals outside protocol fuzzing proper Jiang
171 et al. (2021); Choi et al. (2021); Zhou et al. (2024). This line reinforces our
172 decision to treat the LLM as a bounded proposal mechanism rather than as
173 an unconstrained generator.

174 LLM guidance addresses the synthesis bottleneck more directly. ChatAFL
175 uses RFCs and interaction histories to generate protocol messages Meng et al.
176 (2024). Adjacent systems use LLMs for general fuzz input generation and
177 documentation-guided constraint extraction Xia et al. (2024); Wang et al.
178 (2024); Xie et al. (2022); Wang et al. (2023). Other systems use LLMs for
179 specification-guided device fuzzing or fuzz-driver generation Ma et al. (2024);
180 Lyu et al. (2024). Our work is closest to ChatAFL. The difference is the
181 control boundary. LoopFuzz asks whether a proposal should be admitted,
182 whether its hypothesis survives runtime evidence, and how the IPSM should
183 guide the next query.

184 Benchmarking practice is also part of the context. ProFuzzBench estab-
185 lished a reusable baseline for repeated protocol-fuzzing experiments Natella
186 and Pham (2021). Recent methodology work shows how easily comparisons

187 become unstable without fixed budgets, repeated trials, and careful inter-
188 pretation of coverage signals Schloegel et al. (2024); Böhme et al. (2022);
189 Poncelet et al. (2022). We follow that guidance because runtime-validated
190 assistance is especially sensitive to campaign history.

191 4. LoopFuzz Design

192 Fig. 2 summarizes the three coupled loops in LoopFuzz: the Grammar
193 Hypothesis Loop, the Fuzz Execution Loop, and the LLM Intervention Loop.

194 LoopFuzz preserves the stateful greybox fuzzing execution path. It sets
195 a strict control boundary between LLM generation and queue admission. A
196 model output changes the campaign only after runtime evidence supports ad-
197 mission. The admission rule checks message structure, parseability, response
198 behavior, inferred state reachability, and downstream campaign value. The
199 method has four coupled components: grammar hypothesis generation, run-
200 time admission control, counterexample-guided refinement, and state-aware
201 scheduling.

202 This control boundary is the core idea of the paper. ChatAFL is open-
203 loop: it uses a fixed trigger, a plain-text prompt, and direct admission of the
204 extracted request. LoopFuzz is closed-loop: IPSM growth, state productivity,
205 and hypothesis fitness determine when the LLM is queried, what context it
206 receives, and whether its output is admitted.

207 The three loops in Fig. 2 define the analytical roles of the method. The
208 Grammar Hypothesis Loop constructs and repairs message hypotheses from
209 RFC knowledge, seed traces, and observed responses. The Fuzz Execution
210 Loop performs state selection, mutation, bounded execution, IPSM update,

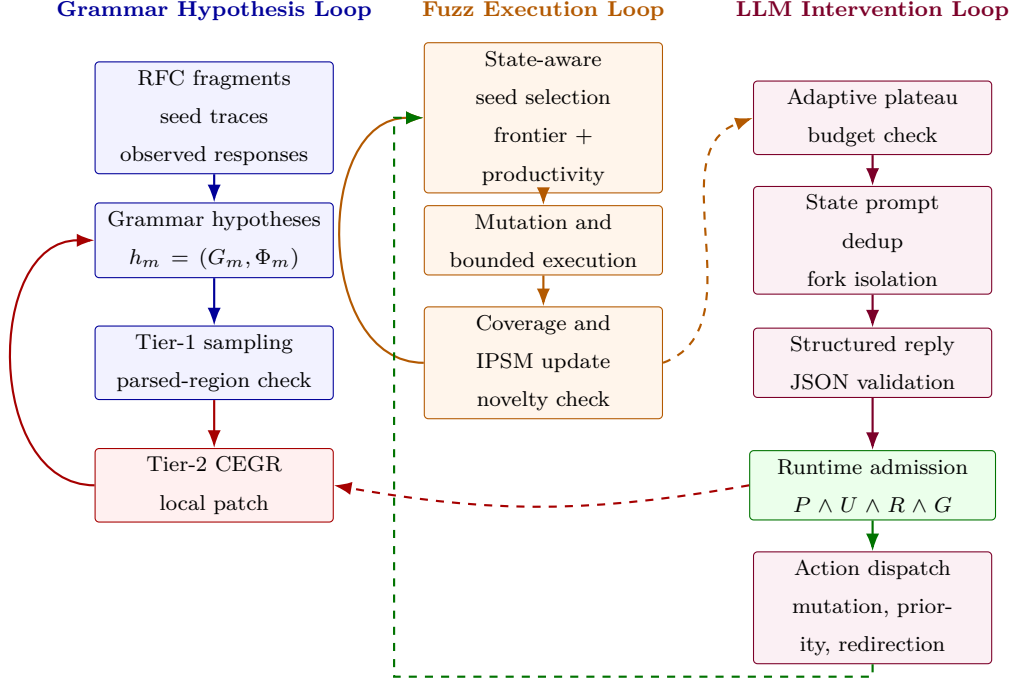


Figure 2: LoopFuzz changes the LLM from a queue producer into a bounded policy proposer. The Grammar Hypothesis Loop maintains message hypotheses, the Fuzz Execution Loop updates coverage and IPSM feedback, and the LLM Intervention Loop proposes bounded interventions; runtime admission checks mediate each proposal before it can affect the queue.

211 and novelty checking. The LLM Intervention Loop is activated only under
 212 plateau conditions and returns either a validated continuation or a bounded
 213 control action.

214 The remaining labels in Fig. 2 denote control interfaces rather than in-
 215 dependent claims. *Tier-1 Sampling* is the cheap online check over parsed
 216 execution regions. *Tier-2 Refinement* is the counterexample-triggered repair
 217 path. *JSON Validation* checks the schema of a model reply before action
 218 dispatch. *Action Dispatch* maps an admitted action to one of three bounded

219 campaign controls: mutation proposal, seed prioritization, or target-state
220 redirection. *Dedup* prevents repeated prompts over the same state summary.
221 Fork-based query isolation bounds failures in the model-query path.

222 4.1. Grammar Hypothesis Loop

223 Let $\mathcal{L}(c, s, \Sigma)$ denote the LLM generation function under prompt context
224 c , state summary s , and protocol knowledge Σ . LoopFuzz uses $\mathcal{L}$ in two
225 stages. During initialization, it preserves ChatAFL’s template extraction
226 and seed enrichment path. Initial enrichment is scheduled as a bounded
227 phase so that corpus diversification does not dominate early campaign time.
228 During fuzzing, LoopFuzz adds a lazily initialized hypothesis layer. This
229 layer is activated at the first plateau event and updated through sampled
230 validation and refinement. The separation is important. The initialization
231 phase broadens the initial seed set. Runtime hypotheses support runtime-
232 validated control during long campaigns.

233 The Grammar Hypothesis Loop draws on three evidence sources when
234 they are available: RFC fragments, queued seed traces or packet examples,
235 and observed server responses, including response codes and error texts. For
236 each message type m , the loop maintains a hypothesis $h_m = (G_m, \Phi_m)$. Here,
237 G_m is an ABNF-style or template-style production, and Φ_m is a set of field
238 constraints. In LoopFuzz, Φ_m captures length bounds, enumerated values,
239 token dependencies, and simple cross-field relations. This representation is
240 more constrained than free-form text and lighter than a full formal protocol
241 specification. We choose it because sampled online validation must remain
242 cheap.

243 4.2. Runtime Admission Controller

244 The runtime admission controller is the central mechanism in our design.
245 It does not formally prove correctness. Instead, it enforces four runtime
246 checks before an LLM contribution can affect the campaign: parseability,
247 acceptability, state reachability, and coverage gain. A candidate intervention
248 x is admitted only if

$$A(x) = P(x) \wedge U(x) \wedge R(x) \wedge G(x),$$

249 where P denotes local parseability, U denotes response-grounded acceptabil-
250 ity, R denotes evidence of new or redirected state reachability, and G denotes
251 downstream gain in IPSM structure or coverage.

252 The FTP example in Fig. 1 illustrates the same checks. The candidate
253 `RETR example_file.txt` can satisfy local syntax under P , but the live 425
254 response makes U fail because no data channel has been negotiated. The
255 failed execution also provides no useful new transition or coverage signal, so
256 R and G do not support admission. LoopFuzz therefore rejects the candidate
257 and records a counterexample that constrains CEGR to repair the missing
258 `PASV/PORT` precondition.

259 4.2.1. Structured Output and Parseability

260 The plateau stage accepts either a concrete next-request candidate or a
261 bounded structured-action object. Runtime validation first checks structural
262 well-formedness, field types, length bounds, and action-schema consistency.
263 LoopFuzz then tests local parseability against the current message hypothe-
264 ses. It reuses parsed message regions already available during ordinary execu-
265 tion. This step rejects structurally invalid interventions before they perturb

266 campaign control.

267 4.2.2. *Response-Grounded Acceptability*

268 Parseability alone is insufficient. A parsed request can still be semanti-
269 cally premature. LoopFuzz therefore classifies the resulting response and
270 treats a non-rejection response as positive evidence. Rejected or persis-
271 tently unproductive responses reduce state productivity and hypothesis fit-
272 ness. This gives an execution-grounded notion of acceptability rather than
273 a semantic proof. The controller asks a narrow question: does live target
274 behavior support admission?

275 4.2.3. *State Reachability and Coverage Gain*

276 The final admission question is whether a candidate helps the campaign.
277 LoopFuzz measures this with the standard AFLNet-style novelty test on new
278 IPSM states, new IPSM transitions, or new edge coverage. The controller
279 therefore uses an IPSM-derived reachability signal, not text quality alone.
280 A hypothesis fragment remains in active use only when execution provides
281 positive evidence in state reachability or coverage. Fluent but repeatedly
282 unproductive candidates are filtered out at the control boundary instead of
283 accumulating as low-value queue bias.

284 4.3. *Counterexample-Guided Refinement*

285 When sampled validation fails repeatedly, LoopFuzz records a concrete
286 counterexample. The record contains the failure-triggering request fragment,
287 the observed response, and the mismatch against the current hypothesis. Re-
288 finement is deliberately local. Instead of rewriting the full protocol descrip-
289 tion, the refinement interface constrains each repair to one field constraint or

290 one production fragment. This single-patch discipline narrows the model’s
 291 search space, limits unconstrained edits, and makes the repair path auditable.
 292 The resulting update is reintegrated into protocol matching and re-evaluated
 293 by later sampled validation.

294 This repair boundary is intentionally weaker than formal grammar repair.
 295 The one-field or one-production constraint is enforced through the refinement
 296 prompt and audit log, not through hard AST differencing. The current
 297 implementation also stores concrete failed traces without counterexample
 298 minimization. We therefore claim bounded local refinement, not complete
 299 protocol inference.

300 4.4. *State-Aware Scheduling and Plateau-Triggered Assistance*

301 Reducing malformed generations solves only half of the problem. Stateful
 302 fuzzers also stagnate when the campaign keeps revisiting locally dense IPSM
 303 regions while failing to cross narrow protocol bridges. LoopFuzz therefore
 304 treats IPSM feedback as a first-class scheduling signal.

305 To address this, LoopFuzz augments state selection with a frontier bonus
 306 β derived from the out-degree of the IPSM:

$$\beta = \begin{cases} 4.0 & \text{if } \deg^+(s) \leq 1 \\ 2.0 & \text{if } \deg^+(s) \leq 3 \\ 1.0 & \text{otherwise} \end{cases}$$

307 States absent from the IPSM receive the maximum bonus $\beta=4$. LoopFuzz
 308 then modulates this bonus with a three-level error-productivity hint: un-
 309 known, likely error, or confirmed productive. The reported implementa-
 310 tion encodes these levels as 0, 1, and 2. This hint downweights frontier

311 states that repeatedly behave like rejection sinks, but it does not discard
 312 them outright. In the reported policy, hint level 1 applies a $\beta \times 0.5$ penalty.
 313 Hint level 0 is reduced to $\beta \times 0.7$ only after repeated low-productivity selec-
 314 tions (*selected_times* ≥ 30 and *productivity* < 0.005). Hint level 2 receives no
 315 penalty. The hint is initialized from response-code classification and updated
 316 online. As a result, states can be promoted or demoted as evidence accu-
 317 mulates rather than being fixed by an early response pattern. This policy
 318 approximates a rare-transition preference without requiring an exact model
 319 of the full protocol state space.

320 These constants should be read as the reported control policy, not as a
 321 claim of global optimality. The three frontier-bonus levels $\{4.0, 2.0, 1.0\}$ sep-
 322 arate unseen or low-out-degree frontier states from denser regions by coarse
 323 preference rather than by fine-grained tuning. The error-productivity penal-
 324 ties and the $(30, 0.005)$ gate reduce repeated investment in response-derived
 325 rejection sinks while still allowing those states to recover if later executions
 326 become productive. Likewise, the $(80, 30\%)$ node-stagnation safeguard and
 327 the 64-call intervention budget are bounded control parameters. They pre-
 328 vent persistent hot-spot lock-in and unbounded LLM overhead. We report
 329 them to make the policy auditable; this study does not claim a full sensitivity
 330 analysis over the entire hyperparameter surface.

331 A complementary graph stagnation check tracks whether IPSM growth
 332 has collapsed at the graph level rather than only at the local state level. When
 333 no new nodes appear for an extended interval, the scheduler intermittently
 334 injects random selection to avoid persistent hot-spot lock-in. In the reported
 335 configuration, that safeguard activates after 80 consecutive rounds without

node growth and uses a 30% random-selection probability.

Plateau detection uses the IPSM edge growth rate $\dot{E}$ (edges gained per minute), recalculated every 60 seconds of wall-clock time, to set an adaptive threshold:

$$\tau(\dot{E}) = \begin{cases} 300 & \text{if } \dot{E} > 5 \\ 200 & \text{if } 1 < \dot{E} \leq 5 \\ 150 & \text{if } \dot{E} \leq 1 \end{cases}$$

The initial value is $\tau=200$ for textual protocols. MQTT uses the higher thresholds $\{500, 400, 300\}$ under the same edge-growth bins because binary framing and broker-mediated behavior expose state progress more sparsely. More generally, LoopFuzz treats plateau triggering as a rate-sensitive control decision, not as a fixed-frequency query rule. Assistance is invoked only when edge growth remains weak and the model-intervention budget is not exhausted. In the reported configuration, the plateau path opens when the unproductive streak reaches τ and prior model interventions remain below 64. Once inside that path, hypotheses are initialized lazily on first use. Refinement remains gated by validation count and accumulated counterexamples. Prompts are deduplicated over the most recent 64 LLM invocations. This prevents assistance from reacting to the same state summary again and again.

The plateau prompt exposes a compact state summary rather than a raw execution log. It includes IPSM node and edge counts, the current edge-growth rate, pending favored seeds, queued paths, the current target state, and the least productive states. The admissible action space is also bounded. A model can redirect the target state, reprioritize a small set of

358 seeds, or propose bounded mutations over an existing seed. These actions
359 are not admitted because they are fluent. They must still pass the same
360 runtime admission boundary. This design keeps the LLM in the role of a
361 state-directed policy proposer.

362 MQTT-specific scheduling is treated as a protocol adaptor, not as the
363 general mechanism of LoopFuzz. For MQTT campaigns, packet-type selec-
364 tion uses a state-indexed action policy, and mutation-family selection uses a
365 bandit-style controller. These signals help expose broker forwarding behav-
366 ior to the IPSM, but the paper reports Mosquitto separately because this
367 adaptor is protocol-specific and because MBFuzzer is a specialized MQTT
368 baseline. In short, assistance is conditional, budgeted, state-directed, and
369 scoped by protocol evidence.

370 Algorithm 1 specifies the plateau path and the hypothesis-refinement
371 gate. It formalizes how state-aware triggering, runtime admission checks,
372 and CEGR interact within one campaign loop.

373 This integration also imposes a bounded operating model. The initial
374 enrichment phase is separated from long-running admission control so that
375 seed completion does not dominate early wall-clock time. Hypothesis con-
376 struction is deferred until the first plateau event. This avoids paying the full
377 refinement cost before the campaign exposes evidence of stagnation. Model
378 queries use process isolation, and network cleanup is bounded to limit persis-
379 tent stalls on forking daemons. These mechanisms are not separate novelty
380 claims. They define the operating conditions under which runtime admission
381 remains compatible with long-running stateful fuzzing.

Algorithm 1: Runtime-Validated Plateau Recovery in LoopFuzz

Input: Target T , LLM function $\mathcal{L}$, Corpus C , Target edge growth $\dot{E}$

Output: Coverage bitmap $\mathcal{B}_{global}$, IPSM $\mathcal{M}$

```
1 Initialize global bitmap  $\mathcal{B}_{global} \leftarrow \emptyset$ ;  
2 Initialize IPSM  $\mathcal{M} \leftarrow \emptyset$ ;  
3 while Campaign Time < 24 hours do  
4    $s \leftarrow \text{SelectSeed}(C, \mathcal{M})$ ;  
5    $\text{trace}_{exec}, Q_{path} \leftarrow \text{MutateAndExecute}(T, s)$ ;  
6    $\text{UpdateIPSM}(\mathcal{M}, Q_{path})$ ;  
7    $\text{UpdateFrontierScores}(\mathcal{M})$ ;  
8   if ShouldSampleValidate() then  
9      $\text{ValidateHypothesesOnParsedRegions}(Q_{path})$ ;  
10  end  
11  if PlateauDetected( $\tau(\dot{E})$ ) then  
12    if HypCtxMissing() then  
13       $\text{InitHypLazily}(C, Q_{path})$ ;  
14    end  
15    if RefineGateOpen() then  
16       $\text{CEGR\_RefineLowFitnessHyp}()$ ;  
17    end  
18     $c_{state} \leftarrow \text{BuildStatePrompt}(\mathcal{M}, C, Q_{path})$ ;  
19    if  $\neg \text{PromptSeen\_Dedup}(c_{state})$  then  
20       $r \leftarrow \text{ForkQueryLLM}(\mathcal{L}, c_{state})$ ;  
21      if  $A(r) \equiv P(r) \wedge U(r) \wedge R(r) \wedge G(r)$  then  
22         $\text{ActionDispatch}(r, C)$ ;  
23      end  
24    end  
25  end  
26 end
```

382 5. Experimental Evaluation Design

383 We organize the evaluation around three research questions that match
384 the paper’s main claims. We ask whether closed-loop admission control
385 changes whole-campaign behavior, whether the evaluated state-aware con-
386 trol policy helps campaigns escape IPSM plateaus, and whether those effects
387 persist in final 24-hour outcomes.

- 388 • **RQ1 (Whole-Campaign Effect):** Does runtime-validated LLM as-
389 sistance prevent the coverage regressions that can arise from direct
390 open-loop generation?
- 391 • **RQ2 (IPSM Escape Dynamics):** Does the evaluated state-aware
392 policy, with frontier scheduling and plateau-triggered intervention, sus-
393 tain IPSM growth after the baselines stall?
- 394 • **RQ3 (End-of-Campaign Outcome):** Under equal 24-hour budgets,
395 how does LoopFuzz affect final `b_abs` coverage and final `edges` relative
396 to prior stateful greybox baselines?

397 5.1. Experimental Setup

398 We ran all experiments on a dedicated Ubuntu 20.04 server with 32 CPU
399 cores and 64GB RAM. The benchmark suite spanned 10 implementations
400 across seven protocol families:

- 401 1. **FTP:** LightFTP, bftpd, ProFTPD, and Pure-FTPd.
- 402 2. **SMTP:** Exim.
- 403 3. **RTSP:** Live555.
- 404 4. **SIP:** Kamailio.

405 5. **DAAP/DACP**: Forked-daapd.

406 6. **HTTP**: Lighttpd1.

407 7. **MQTT**: Mosquitto.

408 The benchmark suite contains 10 targets, but the provided run-summary
409 archive supports different comparisons on different subsets. The primary
410 AFLNet/ChatAFL/**chatafl_opt** comparison uses the nine off-the-shelf non-
411 MQTT targets with all three primary fuzzers: LightFTP, bftpd, ProFTPD,
412 Pure-FTPd, Exim, Live555, Kamailio, Forked-daapd, and Lighttpd1. We
413 retain only completed runs with at least 1400 archived minutes; if a fuzzer-
414 target pair contains more than 10 valid runs, we keep the lowest 10 run IDs.
415 This rule removes one short LightFTP ChatAFL run and trims the extra
416 **chatafl_opt** runs for Kamailio and Forked-daapd.

417 Mosquitto needs separate treatment. In the provided archive, the Mosquitto
418 rows do not form the same 24-hour three-way comparison as the nine primary
419 targets. AFLNet has 10 completed 24-hour rows. The available **chatafl_opt**
420 Mosquitto rows report **b_abs** and **edges**, but their archived runtimes range
421 from 31 to 444 minutes, with a mean of 137.4 minutes. The MBFuzzer direc-
422 tory contains a Mosquitto summary file rather than a standard run-summary
423 file and reports **branch_covered**. We therefore report Mosquitto as a sepa-
424 rately qualified MQTT auxiliary comparison rather than pooling it with the
425 primary 24-hour table.

426 All evaluated subjects are textual or semi-textual request/response pro-
427 tocols. We restrict our claims to this regime. The method has not been
428 validated on opaque binary or encrypted protocols such as DNS, TLS, SSH,
429 DTLS, or DICOM.

Both ChatAFL and LoopFuzz used the same pinned GPT-4o-mini endpoint. Prompts remained protocol-agnostic beyond protocol name, observed state, and runtime summary. We did not hand-specialize prompt text per target. Each campaign ran for 24 hours. The primary table used archived repeated runs from a declared per-target result archive, with no per-system cherry-picking of directories or best runs.

5.2. Baselines, Metrics, and Statistical Protocol

We used two baseline layers. The *primary* baselines were AFLNet as the stateful greybox baseline and ChatAFL as the direct LLM-assisted baseline. LoopFuzz corresponds to the archived `chatafl_opt` rows. The *auxiliary* reproduced baselines were NSFuzz Qin et al. (2023) on eight non-HTTP/non-MQTT targets and MBFuzzer Song et al. (2025) on Mosquitto. For NSFuzz, we compare AFLNet, ChatAFL, `chatafl_opt`, and NSFuzz side by side using `b_abs` and the archived `edges` field. For MBFuzzer, we compare only coverage-like endpoints because MBFuzzer’s summary does not expose `b_abs` or `edges`.

The main dependent variables are the terminal `b_abs` and `edges` fields in each `run_summary.csv`. We treat `b_abs` as the primary coverage endpoint because it is the least system-specific measure in the available summaries. We treat `edges` as a state-exploration diagnostic. For NSFuzz, we report its archived `edges` field as a same-name artifact metric, but we do not treat it as semantically identical to the response-derived IPSM edges maintained by the primary AFLNet-lineage fuzzers.

For each primary target and metric, we summarized selected terminal values with mean and sample SD. We then applied a two-sided exact Mann-

455 Whitney U test to the per-run terminal values. We use the time-series plots
456 to show campaign dynamics, not to claim significance at every timestamp.
457 We also preserved crash artifacts, but this study does not claim novel vulner-
458 abilities because crash triage and root-cause validation are not normalized
459 across the suite.

460 *5.3. Policy-Ablation Design*

461 The ablation design follows the switches present in the provided ablation
462 archive. The archived matrix compares the evaluated **full** controller against
463 four policy perturbations: **wo_refinement**, **wo_frontier**, **wo_hypothesis**,
464 and **wo_adaptive**. This organization reflects the actual archive rather than
465 an idealized orthogonal factorization. We apply the same deterministic run-
466 selection rule as the primary comparison: keep completed runs with at least
467 1400 minutes, and if more than 10 valid runs remain, keep the lowest 10 run
468 IDs.

469 *5.4. Trace Accounting and Cost Reporting*

470 Because LoopFuzz is a closed-loop LLM-in-the-loop system, the evalua-
471 tion must report more than coverage alone. We therefore logged each plateau
472 event with its prompt, raw model reply, and token use. We also logged each
473 refinement event with the affected hypothesis and its counterexamples. The
474 ProFTPD semantic-drift example in Fig. 1 is a concrete instance of this ev-
475 idence pattern: candidate request, target response, admission decision, and
476 refinement target. These logs are sufficient to recover target-level admission
477 statistics such as parseability pass rate, response-grounded acceptance rate,
478 and CEGR repair outcomes. Those summaries are not normalized across

Table 2: Primary end-of-campaign results from `run_summary.csv` (mean $\pm$ sample SD over selected completed runs). Bold marks the highest mean among AFLNet, ChatAFL, and LoopFuzz. * marks LoopFuzz `b_abs` wins significant versus both primary baselines under a two-sided exact Mann-Whitney U test at $p < 0.05$; $\dagger$ marks LoopFuzz `edges` wins significant versus both primary baselines. All rows use 10 runs per fuzzer except LightFTP ChatAFL, where one short run is excluded and $n = 9$.

Proto.	Target	Final <code>b_abs</code>			Final <code>edges</code>		
		AFLNet	ChatAFL	LoopFuzz	AFLNet	ChatAFL	LoopFuzz
FTP	LightFTP	72.8$\pm$4.7	71.3 $\pm$ 1.0	71.1 $\pm$ 0.3	182.7 $\pm$ 11.9	199.4 $\pm$ 14.1	206.6$\pm$13.0
FTP	bftpd	488.6 $\pm$ 13.9	492.3 $\pm$ 15.1	501.5$\pm$7.4	166.9 $\pm$ 9.4	205.2 $\pm$ 9.7	221.7$\pm$16.9$\dagger$
FTP	ProFTPD	4919.9 $\pm$ 97.8	5317.3 $\pm$ 101.7	5358.8$\pm$136.1	180.5 $\pm$ 23.1	250.5 $\pm$ 9.1	271.7$\pm$18.8$\dagger$
FTP	Pure-FTPD	1054.3 $\pm$ 76.4	1179.2 $\pm$ 31.0	1202.0$\pm$96.2*	213.9 $\pm$ 11.9	266.2 $\pm$ 14.3	287.6$\pm$29.5$\dagger$
SMTP	Exim	3794.5 $\pm$ 42.4	3746.2 $\pm$ 47.7	3866.6$\pm$86.0	88.3 $\pm$ 5.3	116.4 $\pm$ 10.2	125.5$\pm$8.5
RTSP	Live555	2894.1 $\pm$ 23.3	2911.1 $\pm$ 10.9	3088.7$\pm$22.4*	103.9 $\pm$ 10.1	151.2 $\pm$ 3.9	158.7$\pm$8.5$\dagger$
SIP	Kamailio	10244.0 $\pm$ 118.1	10414.7$\pm$143.4	10246.8 $\pm$ 274.5	101.1 $\pm$ 7.7	122.4 $\pm$ 12.1	130.6$\pm$8.5
DAAP/DACP	Forked-daapd	2326.3$\pm$77.3	2315.3 $\pm$ 50.1	2086.1 $\pm$ 204.7	21.1 $\pm$ 1.3	20.5 $\pm$ 2.1	22.3$\pm$4.3
HTTP	Lighttpd1	1930.4 $\pm$ 18.7	1987.5 $\pm$ 38.3	2156.5$\pm$111.8*	14.9 $\pm$ 3.8	29.7 $\pm$ 4.7	32.5$\pm$10.8

the full archive, so we report them as audit artifacts and as a limitation rather than as aggregate evidence. As a result, semantic drift remains a motivating failure mode and an auditable trace category here, not a quantified cross-target headline metric.

The four data directories audited here do not contain a normalized cross-target LLM-cost summary. We therefore do not make a cost-efficiency claim in this re-audited experimental section.

6. Results

Table 2 summarizes the terminal aggregates, and the endpoint figures visualize the same selected-run comparison.

Table 3: Auxiliary four-way NSFuzz comparison from `run_summary.csv`. Bold marks the highest mean within each row and metric. NSFuzz `edges` are reported as an archived same-name artifact field, not as a guaranteed semantically identical IPSM abstraction. All rows use the selected-run rule; LightFTP ChatAFL has $n = 9$, and the other cells have $n = 10$.

Target	Final <code>b_abs</code>				Final <code>edges</code>			
	AFLNet	ChatAFL	LoopFuzz	NSFuzz	AFLNet	ChatAFL	LoopFuzz	NSFuzz
LightFTP	72.8±4.7	71.3±1.0	71.1±0.3	414.3±9.8	182.7±11.9	199.4±14.1	206.6±13.0	11.0±0.0
bftpd	488.6±13.9	492.3±15.1	501.5±7.4	477.6±3.9	166.9±9.4	205.2±9.7	221.7±16.9	48.7±4.8
ProFTPD	4919.9±97.8	5317.3±101.7	5358.8±136.1	5335.0±86.9	180.5±23.1	250.5±9.1	271.7±18.8	186.0±11.5
Pure-FTPD	1054.3±76.4	1179.2±31.0	1202.0±96.2	1332.2±8.8	213.9±11.9	266.2±14.3	287.6±29.5	17.6±0.5
Exim	3794.5±42.4	3746.2±47.7	3866.6±86.0	4191.5±62.0	88.3±5.3	116.4±10.2	125.5±8.5	13.9±1.2
Live555	2894.1±23.3	2911.1±10.9	3088.7±22.4	3021.5±42.1	103.9±10.1	151.2±3.9	158.7±8.5	49.8±2.0
Kamailio	10244.0±118.1	10414.7±143.4	10246.8±274.5	10635.3±123.8	101.1±7.7	122.4±12.1	130.6±8.5	14.3±0.7
Forked-daapd	2326.3±77.3	2315.3±50.1	2086.1±204.7	2722.8±44.6	21.1±1.3	20.5±2.1	22.3±4.3	27.0±6.1

Table 4: MQTT auxiliary comparison. AFLNet and `chatafl_opt` are taken from the provided Mosquitto `run_summary.csv`; MBFuzzer is taken from its Mosquitto `summary.csv`. This is not a same-budget or same-metric comparison: `chatafl_opt` runs are much shorter, and MBFuzzer reports `branch_covered` rather than `b_abs`.

System	Runs	Runtime (min)	Coverage-like endpoint	State endpoint
AFLNet	10	1479.0±0.0	<code>b_abs</code> : 1814.3±147.3	<code>edges</code> : 36.0±4.5
LoopFuzz (<code>chatafl_opt</code>)	10	137.4±134.7	<code>b_abs</code> : 2518.5±269.9	<code>edges</code> : 107.7±18.1
MBFuzzer	10	1480.0±0.0	<code>branch_covered</code> : 2827.0±58.9	not reported

6.1. RQ1: Whole-Campaign Effect

LoopFuzz is evaluated here as an integrated control policy, not as an isolated rejection heuristic. Table 2 shows that direct open-loop assistance is not uniformly robust under the selected archived runs. ChatAFL finishes below AFLNet on LightFTP and Exim in `b_abs`, is close to AFLNet on Forked-daapd, and improves several other targets. Under these conditions, improved generation alone does not consistently translate into stronger end-of-campaign coverage.

497 Across the nine primary targets, LoopFuzz reaches the highest mean fi-
 498 nal **b_abs** on six targets: bftpd, ProFTPD, Pure-FTPd, Exim, Live555, and
 499 Lighttpd1. AFLNet remains highest on LightFTP and Forked-daapd, while
 500 ChatAFL remains highest on Kamilio. Exact Mann-Whitney testing nar-
 501 rows the strongest **b_abs** evidence to Pure-FTPd, Live555, and Lighttpd1.
 502 The robust coverage claim supported by the audited summaries is therefore
 503 bounded: LoopFuzz improves the closest AFLNet/ChatAFL lineage on most,
 504 but not all, primary targets.

505 Table 3 shows that the broader NSFuzz baseline picture is mixed. Across
 506 the eight NSFuzz targets, NSFuzz has the highest **b_abs** on LightFTP, Pure-
 507 FTPd, Exim, Kamilio, and Forked-daapd. LoopFuzz has the highest **b_abs**
 508 on bftpd, ProFTPD, and Live555. For reported **edges**, LoopFuzz is highest
 509 on seven targets, while NSFuzz is highest on Forked-daapd. Table 4 sepa-
 510 rately reports MQTT: the available **chatafl_opt** Mosquitto rows have higher
 511 **b_abs** and **edges** than the available AFLNet rows, but they are much shorter;
 512 MBFuzzer reports higher coverage than both in its own **branch_covered** field
 513 under 24-hour runs.

514 6.2. RQ2: IPSM Escape Dynamics

515 Fig. 3 reports the final **edges** comparison for the protocol groups in the
 516 main quantitative view.

517 On terminal **edges**, LoopFuzz attains the highest mean on all nine pri-
 518 mary targets. The strongest same-target evidence appears on bftpd, ProFTPD,
 519 Pure-FTPd, and Live555, where the LoopFuzz **edges** win is significant versus
 520 both AFLNet and ChatAFL. LightFTP, Exim, Kamilio, Forked-daapd, and
 521 Lighttpd1 remain positive but descriptive against at least one primary base-

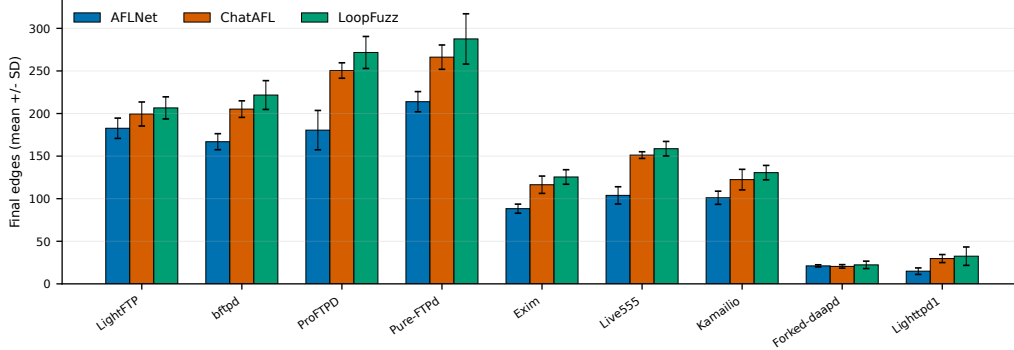


Figure 3: Final `edges` comparison for the nine primary targets, recomputed from `run_summary.csv` using the selected-run rule.

line. The NSFuzz auxiliary rows point in the opposite direction on Forked-
daapd, where NSFuzz reports higher `edges`; on the other seven NSFuzz tar-
gets, LoopFuzz has higher reported `edges`.

6.3. RQ3: End-of-Campaign Outcome

Fig. 4 reports the corresponding final `b_abs` comparison for the same
quantitative-view layout.

The `b_abs` comparison is less uniform than the `edges` comparison. Loop-
Fuzz is clearly strongest on Live555 and Lighttpd1, remains ahead on bftpd,
ProFTPD, Pure-FTPd, and Exim, but does not lead LightFTP, Kamailio,
or Forked-daapd. This distinction matters for interpretation: the audited
archive supports a strong state-exploration result and a more selective cov-
erage result, not a blanket dominance claim.

6.4. Ablation: Policy Perturbations

Table 5 reports the archived policy ablations as deltas relative to the eval-
uated `full` controller. Each cell shows $\Delta b_abs / \Delta edges$. The pattern is

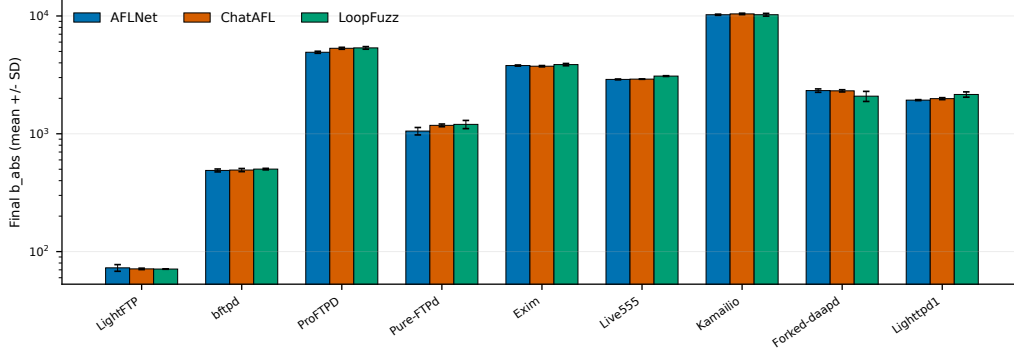


Figure 4: Final **b_abs** comparison for the nine primary targets, recomputed from `run_summary.csv` using the selected-run rule.

537 mixed rather than monotone. Removing refinement improves **b_abs** on sev-
 538 eral targets, including ProFTPD (+100.8), Kamailio (+97.6), and Forked-
 539 daapd (+90.2), but reduces **edges** on bftpd, ProFTPD, Live555, Forked-
 540 daapd, and Lighttpd1. Removing frontier control helps **b_abs** on ProFTPD
 541 (+122.0), Pure-FTPd (+55.0), Kamailio (+87.0), and several others, while
 542 reducing **edges** on bftpd, Live555, and Forked-daapd. The audited ablations
 543 therefore do not support a claim that every exposed sub-control is individu-
 544 ally necessary across all targets.

545 The strongest lesson from the ablation archive is that controller switches
 546 interact with target behavior. `wo_hypothesis` and `wo_adaptive` can im-
 547 prove **b_abs** on some targets but degrade **edges** on others. This reinforces
 548 an integrated-policy interpretation rather than a clean per-module causal
 549 decomposition.

Table 5: Policy-ablation results from `ten_groups_ablation_ten` relative to the evaluated `full` controller. The first number in each cell is final `b_abs`; the second is final `edges`. The `Full` column reports mean final `b_abs` / `edges`; other cells report deltas relative to that mean. Runs with archived runtime below 1400 minutes are excluded; when more than 10 valid runs remain, we keep the lowest 10 run IDs.

Target	Full	wo_refinement	wo_frontier	wo_hypothesis	wo_adaptive
LightFTP	71.1 / 206.6	+7.9 / +0.2	+0.1 / +1.6	+3.9 / -1.0	+0.8 / +5.8
bftpd	501.5 / 221.7	-3.1 / -7.5	-3.0 / -8.4	-5.2 / -5.0	-7.0 / -12.1
ProFTPD	5358.8 / 271.7	+100.8 / -4.1	+122.0 / +6.6	-17.5 / -1.3	-12.4 / +1.0
Pure-FTPd	1202.0 / 287.6	+25.6 / +0.6	+55.0 / +27.4	+62.1 / +24.5	+46.9 / +14.1
Exim	3866.6 / 125.5	+37.4 / +3.9	+22.3 / +3.3	+19.1 / +6.7	+26.9 / +3.4
Live555	3088.7 / 158.7	+1.2 / -1.5	+13.7 / -6.2	-12.2 / -4.4	+5.0 / -3.6
Kamailio	10246.8 / 130.6	+97.6 / +6.3	+87.0 / +3.2	+62.3 / +5.5	+223.2 / +11.6
Forked-daapd	2086.1 / 22.3	+90.2 / -1.5	+20.1 / -0.6	+95.7 / +0.3	+26.9 / -2.1
Lighttpd1	2156.5 / 32.5	-62.4 / -3.3	-15.9 / +1.9	-33.9 / -2.4	-27.1 / +4.1

7. Discussion

The paper makes one central claim. In stateful protocol fuzzing, LLM generation is useful only when it is governed by a strict admission boundary. ChatAFL already showed that an LLM can generate protocol text Meng et al. (2024). LoopFuzz asks a different question: when should such output be trusted by the campaign? Our answer is closed-loop runtime admission control. It checks structure, parseability, response-grounded acceptability, state reachability, and downstream gain before an LLM suggestion can strengthen the queue. This changes the role of the LLM from queue producer to hypothesis proposer.

The admission controller and the scheduler solve different parts of the same problem. Admission control filters low-value suggestions. It does not

562 decide where to spend the recovered budget. State-aware scheduling decides
563 where to search. Without closed-loop intervention, however, it still depends
564 on ordinary mutation to cross semantic bottlenecks. LoopFuzz combines the
565 two because admission control and state-directed search reinforce each other.

566 The results also show why response-derived state counts must be read
567 carefully. A campaign can accumulate superficially different rejection paths
568 without moving toward high-value logic. LoopFuzz therefore reads state ex-
569 ploration together with `b_abs` coverage and demotes states that remain un-
570 productive over repeated selections. In this setting, productive states matter
571 more than raw state counts. For that reason, `edges` is best interpreted as a
572 companion diagnostic for campaign behavior, not as a substitute for `b_abs`.

573 The present method is evaluated on textual or semi-textual protocols
574 whose message regions can be parsed cheaply and whose responses provide
575 useful progress signals. That scope covers the benchmark suite used here
576 and many request-response protocols of practical interest. Opaque binary
577 or encrypted protocols require stronger parsers, auxiliary instrumentation,
578 or richer response abstractions. Response codes alone may not expose state
579 reachability in those settings. That extension matters, but it lies beyond the
580 current empirical scope.

581 The present evaluation is also centered on campaign-control behavior
582 rather than on vulnerability yield. We preserve crashes and queue artifacts,
583 but we do not report new CVEs or newly confirmed bugs in this study be-
584 cause crash deduplication, exploitability review, and cross-target triage re-
585 main incomplete under one uniform protocol. The paper should therefore be
586 read as evidence about campaign control and exploration quality, not as an

587 end-to-end demonstration of stronger bug-finding yield.

588 Counterexample-guided local refinement is also a deliberate design choice.
589 Repeated unconstrained regeneration does not accumulate persistent knowl-
590 edge. Local refinement turns failed executions into stored evidence. It makes
591 runtime evidence cumulative rather than disposable. At the same time, the
592 ablation table shows that the exposed refinement switch is not uniformly
593 decisive on every target. The novelty therefore lies in the control interface
594 between model output and system execution, not in prompt wording alone.

595 8. Threats to Validity

596 **Internal validity:** LLM-assisted fuzzing remains sensitive to generation
597 variance, target nondeterminism, and campaign-level timing effects. We mit-
598 igate these threats by fixing the execution environment, pinning the model
599 endpoint, using low temperatures for grammar extraction and refinement,
600 and designing the campaign matrix around repeated 24-hour runs. Even so,
601 plateau triggering and hypothesis refinement are history-dependent. Early
602 differences can propagate into late-campaign variance. The study also can-
603 not cleanly separate model variance from control-policy variance because
604 both act within the same long-running campaigns.

605 **External validity:** The benchmark suite spans 10 implementations
606 across seven protocol families, but the audited primary three-way compari-
607 son covers the nine off-the-shelf non-MQTT targets that contain compara-
608 ble AFLNet, ChatAFL, and `chatafl_opt` rows. The evidence is therefore
609 strongest for textual or semi-textual protocols whose progress is visible in re-
610 sponses. We do not claim unchanged transfer to opaque binary or encrypted

611 protocols without stronger format-specific adaptors. We include auxiliary
612 NSFuzz comparisons on eight targets and an MQTT comparison involv-
613 ing AFLNet, `chatafl_opt`, and MBFuzzer, but those rows are target- and
614 artifact-specific rather than a unified cross-system state-machine comparison.

615 **Construct validity:** We use final `b_abs` and final `edges` as the main
616 proxies for fuzzing effectiveness in the audited summaries. These are stan-
617 dard archive-level measurements, but neither fully captures semantic protocol
618 progress. Our closed-loop runtime admission boundary refers to structured
619 output checks, sampled hypothesis checks, response-grounded acceptability,
620 state-reachability signals, and counterexample-guided refinement. It does
621 not mean formal verification or a perfect pre-execution oracle. We also do
622 not currently report cross-target admission summaries such as parseability
623 pass rate, response-grounded acceptance rate, or CEGR repair success rate.
624 The motivating semantic-drift narrative is therefore operationalized through
625 logs rather than reduced to one global prevalence number.

626 **Reproducibility validity:** We reduce fairness ambiguity where possi-
627 ble. ChatAFL and LoopFuzz share the same pinned model endpoint. Each
628 primary target is drawn from one declared result archive, and the paper
629 now states the deterministic run-selection rule used to handle short or extra
630 rows. The provided Mosquitto rows do not support the same 24-hour three-
631 way comparison because ChatAFL is absent and the available `chatafl_opt`
632 runs are substantially shorter than AFLNet and MBFuzzer. The ablation
633 table is likewise an archive-level reconstruction under the same deterministic
634 selection rule rather than a hand-picked replacement set.

635 9. Conclusion

636 We have shown that fluent protocol text is insufficient for a stateful cam-
637 paign. What matters is a closed-loop runtime admission boundary that can
638 reject semantically premature generations and refine them when they fail.

639 Under the audited primary comparison against AFLNet and ChatAFL,
640 LoopFuzz demonstrates a stronger **edges** profile and a selective **b_abs** ad-
641 vantage. On the nine off-the-shelf targets, it achieves the highest mean final
642 **edges** on all nine targets and the highest mean final **b_abs** on six. The
643 strongest same-target **b_abs** evidence appears on Pure-FTPd, Live555, and
644 Lighttpd1; the strongest **edges** evidence appears on bftpd, ProFTPD, Pure-
645 FTPd, and Live555.

646 The broader baseline picture is mixed. NSFuzz remains strongest on fi-
647 nal **b_abs** for five of the eight reproduced NSFuzz targets, while LoopFuzz
648 has the highest reported **edges** on seven of those eight. On MQTT, the
649 available **chatafl_opt** rows exceed AFLNet in **b_abs** and **edges**, but those
650 rows are much shorter; MBFuzzer reports the highest coverage-like endpoint
651 through **branch_covered** under 24-hour runs. The archived ablations also
652 do not support a claim that every exposed LoopFuzz sub-control is individ-
653 ually necessary across all targets. The evidence therefore supports a precise
654 conclusion: closed-loop runtime admission control improves state-exploration
655 behavior within the closest AFLNet/ChatAFL lineage and improves **b_abs**
656 on most primary targets, but this study does not establish dominance over
657 specialized external fuzzers.

658 **Declaration of Generative AI and AI-assisted technologies in the**
659 **writing process**

660 During the preparation of this work, the authors used OpenAI Codex for
661 language editing, LaTeX formatting, and submission-compliance checking.
662 After using this tool, the authors reviewed and edited the content as needed
663 and take full responsibility for the content of the publication.

664 **References**

- 665 Aafer, Y., You, W., Sun, Y., Shi, Y., Zhang, X., Yin, H., 2021. Android
666 smarttvs vulnerability discovery via log-guided fuzzing, in: 30th USENIX
667 Security Symposium (USENIX Security 2021), pp. 2759–2776.
- 668 Andronidis, A., Cadar, C., 2022. SnapFuzz: high-throughput fuzzing of
669 network applications, in: Proceedings of the 31st ACM SIGSOFT In-
670 ternational Symposium on Software Testing and Analysis, pp. 340–351.
671 doi:10.1145/3533767.3534376.
- 672 Ba, J., Böhme, M., Mirzamomen, Z., Roychoudhury, A., 2022a. Stateful
673 greybox fuzzing, in: 31st USENIX Security Symposium (USENIX Security
674 2022), pp. 3255–3272.
- 675 Ba, J., Duck, G.J., Roychoudhury, A., 2022b. Efficient greybox fuzzing
676 to detect memory errors, in: Proceedings of the 37th IEEE/ACM In-
677 ternational Conference on Automated Software Engineering, pp. 1–12.
678 doi:10.1145/3551349.3561161.

- 679 Beckmann, M., Steffan, J., 2023. Coverage-guided fuzzing of embedded
680 systems leveraging hardware tracing, in: Computer Security – ESORICS
681 2023, pp. 362–378. doi:10.1007/978-3-031-25460-4_21.
- 682 Böhme, M., Pham, V.T., Nguyen, M.D., Roychoudhury, A., 2017. Directed
683 greybox fuzzing, in: Proceedings of the 2017 ACM SIGSAC Conference
684 on Computer and Communications Security, pp. 2329–2344. doi:10.1145/
685 3133956.3134020.
- 686 Bulekov, A., Das, B., Hajnoczi, S., Egele, M., 2023. No grammar, no prob-
687 lem: Towards fuzzing the Linux kernel without system-call descriptions,
688 in: 30th Annual Network and Distributed System Security Symposium
689 (NDSS 2023). doi:10.14722/ndss.2023.24688.
- 690 Böhme, M., Szekeres, L., Metzman, J., 2022. On the reliability of coverage-
691 based fuzzer benchmarking, in: Proceedings of the 44th International Con-
692 ference on Software Engineering, pp. 1621–1633. doi:10.1145/3510003.
693 3510230.
- 694 Chen, Y., Zhong, R., Hu, H., Zhang, H., Yang, Y., Wu, D., Lee, W., 2021.
695 One Engine to Fuzz ’em All: Generic language processor testing with
696 semantic validation, in: 2021 IEEE Symposium on Security and Privacy
697 (SP), pp. 642–658. doi:10.1109/SP40001.2021.00071.
- 698 Choi, J., Kim, D., Kim, S., Grieco, G., Groce, A., Cha, S.K., 2021. SMAR-
699 TIAN: Enhancing smart contract fuzzing with static and dynamic data-
700 flow analyses, in: Proceedings of the 36th IEEE/ACM International Con-

701 ference on Automated Software Engineering, pp. 227–239. doi:10.1109/
702 ASE51524.2021.9678888.

703 Deng, Y., Xia, C.S., Peng, H., Yang, C., Zhang, L., 2023. Large lan-
704 guage models are zero-shot fuzzers: Fuzzing deep-learning libraries via
705 large language models, in: Proceedings of the 32nd ACM SIGSOFT In-
706 ternational Symposium on Software Testing and Analysis, pp. 423–435.
707 doi:10.1145/3597926.3598067.

708 Deng, Y., Xia, C.S., Yang, C., Zhang, S., Yang, S., Zhang, L., 2024. Large
709 language models are edge-case generators: Crafting unusual programs for
710 fuzzing deep learning libraries, in: Proceedings of the 46th IEEE/ACM
711 International Conference on Software Engineering, pp. 70:1–70:13. doi:10.
712 1145/3597503.3623343.

713 Dong, Y., Rashid, S.M.M., Ranjbar, A., Wu, T., Hussain, S.R., Mahmud,
714 M.S., Ishtiaq, A.A., Tu, K., Yang, T., 2025. CoreCrisis: Threat-guided and
715 context-aware iterative learning and fuzzing of 5g core networks, in: 34th
716 USENIX Security Symposium (USENIX Security 2025), pp. 5287–5306.

717 Du, Z., Li, Y., Liu, Y., Mao, B., 2022. WindRanger: A directed grey-
718 box fuzzer driven by deviation basic blocks, in: Proceedings of the
719 44th International Conference on Software Engineering, pp. 2440–2451.
720 doi:10.1145/3510003.3510197.

721 Feng, X., Sun, R., Zhu, X., Xue, M., Wen, S., Liu, D., Nepal, S., Xiang,
722 Y., 2021. Snipuzz: Black-box fuzzing of IoT firmware via message snippet
723 inference, in: Proceedings of the 2021 ACM SIGSAC Conference on Com-

puter and Communications Security, pp. 337–350. doi:10.1145/3460120.
3484543.

Fioraldi, A., D’Elia, D.C., Balzarotti, D., 2021. The use of likely invariants as feedback for fuzzers, in: 30th USENIX Security Symposium (USENIX Security 2021), pp. 2829–2846.

Fiterau-Brostean, P., Jonsson, B., Sagonas, K., Taquist, F., 2022. DTLS-Fuzzer: A DTLS protocol state fuzzer, in: 2022 IEEE Conference on Software Testing, Verification and Validation (ICST), pp. 456–458. doi:10.1109/icst53961.2022.00051.

Fu, J., Liang, J., Wu, Z., Wang, M., Jiang, Y., 2022. Griffin: Grammar-free dbms fuzzing, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–12. doi:10.1145/3551349.3560431.

Herrera, A., Gunadi, H., Magrath, S., Norrish, M., Payer, M., Hosking, A.L., 2021. Seed selection for successful fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 230–243. doi:10.1145/3460319.3464829.

Hu, F., Ji, J., Shu, H., Li, Z., Liu, T., Zhang, C., 2024. Formatted stateful greybox fuzzing of TLS server, in: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), pp. 151–160. doi:10.1109/icst60714.2024.00022.

Jiang, J., Xu, H., Zhou, Y., 2021. RULF: Rust library fuzzing via api dependency graph traversal, in: Proceedings of the 36th IEEE/ACM In-

ternational Conference on Automated Software Engineering, pp. 581–592.
doi:10.1109/ASE51524.2021.9678813.

Kim, H., Ozmen, M.O., Bianchi, A., Celik, Z.B., Xu, D., 2021. PGFUZZ: Policy-guided fuzzing for robotic vehicles, in: 28th Annual Network and Distributed System Security Symposium (NDSS 2021). doi:10.14722/ndss.2021.24096.

Kim, K., Kim, T., Warraich, E., Lee, B., Butler, K.R.B., Bianchi, A., Tian, D.J., 2022. FuzzUSB: Hybrid stateful fuzzing of USB gadget stacks, in: 2022 IEEE Symposium on Security and Privacy (SP), pp. 2212–2229. doi:10.1109/sp46214.2022.9833593.

Lin, P., Wang, P., Zhou, X., Xie, W., Ren, X., Lu, K., 2025. When control flows deviate: Directed grey-box fuzzing with probabilistic reachability analysis, in: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering. doi:10.1109/ASE63991.2025.00065.

Luo, Z., Yu, J., Zuo, F., Liu, J., Jiang, Y., Chen, T., Roychoudhury, A., Sun, J., 2023. Bleem: Packet sequence oriented fuzzing for protocol implementations, in: 32nd USENIX Security Symposium (USENIX Security 2023), pp. 4481–4498.

Lyu, Y., Xie, Y., Chen, P., Chen, H., 2024. Prompt fuzzing for fuzz driver generation, in: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, pp. 3793–3807. doi:10.1145/3658644.3670396.

769 Ma, X., Luo, L., Zeng, Q., 2024. From one thousand pages of specification
770 to unveiling hidden bugs: Large language model assisted fuzzing of matter
771 IoT devices, in: 33rd USENIX Security Symposium (USENIX Security
772 2024), pp. 4783–4800.

773 Meng, R., Mirchev, M., B"ohme, M., Roychoudhury, A., 2024. Large lan-
774 guage model guided protocol fuzzing, in: 31st Annual Network and Dis-
775 tributed System Security Symposium (NDSS 2024). doi:10.14722/ndss.
776 2024.24556.

777 Myung, C., Lee, G., Lee, B., 2022. MundoFuzz: Hypervisor fuzzing with sta-
778 tistical coverage testing and grammar inference, in: 31st USENIX Security
779 Symposium (USENIX Security 2022), pp. 1257–1274.

780 Natella, R., 2022. StateAFL: Greybox fuzzing for stateful network servers.
781 Empirical Software Engineering 27. doi:10.1007/s10664-022-10233-3.
782 art. no. 191.

783 Natella, R., Pham, V.T., 2021. ProFuzzBench: A benchmark for stateful
784 protocol fuzzing, in: Proceedings of the 30th ACM SIGSOFT International
785 Symposium on Software Testing and Analysis, pp. 662–665. doi:10.1145/
786 3460319.3469077.

787 Pham, V.T., B"ohme, M., Roychoudhury, A., 2020. AFLNet: A greybox
788 fuzzer for network protocols, in: 2020 IEEE 13th International Confer-
789 ence on Software Testing, Verification and Validation (ICST), pp. 460–465.
790 doi:10.1109/ICST46399.2020.00062.

791 Poncelet, C., Sagonas, K.K., Tsiftes, N., 2022. So many fuzzers, so little
792 time: Experience from evaluating fuzzers on the contiki-ng network stack,
793 in: Proceedings of the 37th IEEE/ACM International Conference on Au-
794 tomated Software Engineering, pp. 1–13. doi:10.1145/3551349.3556946.

795 Qin, S., Hu, F., Ma, Z., Zhao, B., Yin, T., Zhang, C., 2023. NSFuzz: Towards
796 efficient and state-aware network service fuzzing. ACM Transactions on
797 Software Engineering and Methodology 32, 160:1–160:26. doi:10.1145/
798 3580598.

799 Redini, N., Continella, A., Das, D., Pasquale, G.D., Spahn, N., Machiry, A.,
800 Bianchi, A., Kruegel, C., Vigna, G., 2021. DIANE: Identifying fuzzing
801 triggers in apps to generate under-constrained inputs for iot devices, in:
802 2021 IEEE Symposium on Security and Privacy (SP), pp. 484–500. doi:10.
803 1109/SP40001.2021.00066.

804 Ryu, D., Kim, G., Lee, D., Kim, S., Bae, S., Rhee, J., Kim, T., 2024. Differ-
805 ential fuzzing for data distribution service programs with dynamic config-
806 uration, in: Proceedings of the 39th IEEE/ACM International Conference
807 on Automated Software Engineering, pp. 807–818. doi:10.1145/3691620.
808 3695073.

809 Sagonas, K., Typaldos, T., 2023. EDHOC-Fuzzer: An EDHOC protocol
810 state fuzzer, in: Proceedings of the 32nd ACM SIGSOFT International
811 Symposium on Software Testing and Analysis, pp. 1495–1498. doi:10.
812 1145/3597926.3604922.

813 Salls, C., Jindal, C., Corina, J., Kruegel, C., Vigna, G., 2021. Token-level

814 fuzzing, in: 30th USENIX Security Symposium (USENIX Security 2021),
815 pp. 2795–2809.

816 Scharnowski, T., Bars, N., Schloegel, M., Gustafson, E., Muench, M., Vigna,
817 G., Kruegel, C., Holz, T., Abbasi, A., 2022. Fuzzware: Using precise
818 MMIO modeling for effective firmware fuzzing, in: 31st USENIX Security
819 Symposium (USENIX Security 2022), pp. 1239–1256.

820 Schloegel, M., Bars, N., Schiller, N., Bernhard, L., Scharnowski, T., Crump,
821 A., Ale-Ebrahim, A., Bissantz, N., Muench, M., Holz, T., 2024. SoK:
822 Prudent evaluation practices for fuzzing, in: 2024 IEEE Symposium on
823 Security and Privacy (SP), pp. 1974–1993. doi:10.1109/sp54263.2024.
824 00137.

825 Song, X., Wu, J., Zeng, Y., Pan, H., Zuo, C., Zhao, Q., Guo, S., 2025.
826 MBFuzzer: A multi-party protocol fuzzer for MQTT brokers, in: 34th
827 USENIX Security Symposium (USENIX Security 2025), pp. 6179–6197.

828 Srivastava, P., Payer, M., 2021. Gramatron: effective grammar-aware
829 fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Sym-
830 posium on Software Testing and Analysis, pp. 244–256. doi:10.1145/
831 3460319.3464814.

832 Trippel, T., Shin, K.G., Chernyakhovsky, A., Kelly, G., Rizzo, D., Hicks, M.,
833 2022. Fuzzing hardware like software, in: 31st USENIX Security Sympo-
834 sium (USENIX Security 2022), pp. 3237–3254.

835 Wang, D., Li, Y., Zhang, Z., Chen, K., 2023. CarpetFuzz: Automatic
836 program option constraint extraction from documentation for fuzzing, in:

837 32nd USENIX Security Symposium (USENIX Security 2023), pp. 1919–
838 1936.

839 Wang, J., Yu, L., Luo, X., 2024. LLMIF: Augmented large language model
840 for fuzzing IoT devices, in: 2024 IEEE Symposium on Security and Privacy
841 (SP), pp. 881–896. doi:10.1109/sp54263.2024.00211.

842 Wu, F., Luo, Z., Zhao, Y., Du, Q., Yu, J., Peng, R., Shi, H., Jiang, Y., 2024.
843 Logos: Log guided fuzzing for protocol implementations, in: Proceedings
844 of the 33rd ACM SIGSOFT International Symposium on Software Testing
845 and Analysis, pp. 1720–1732. doi:10.1145/3650212.3680394.

846 Wu, M., Jiang, L., Xiang, J., Huang, Y., Cui, H., Zhang, L., Zhang, Y.,
847 2022. One fuzzing strategy to rule them all, in: Proceedings of the 44th
848 International Conference on Software Engineering, pp. 1634–1645. doi:10.
849 1145/3510003.3510174.

850 Xia, C.S., Paltenghi, M., Tian, J.L., Pradel, M., Zhang, L., 2024. Fuzz4All:
851 Universal fuzzing with large language models, in: Proceedings of the
852 IEEE/ACM 46th International Conference on Software Engineering, pp.
853 1–13. doi:10.1145/3597503.3639121.

854 Xie, D., Li, Y., Kim, M., Pham, H.V., Tan, L., Zhang, X., Godfrey, M.W.,
855 2022. DocTer: documentation-guided fuzzing for testing deep learning
856 API functions, in: Proceedings of the 31st ACM SIGSOFT International
857 Symposium on Software Testing and Analysis, pp. 176–188. doi:10.1145/
858 3533767.3534220.

859 Zhang, C., Lin, X., Li, Y., Xue, Y., Xie, J., Chen, H., Ying, X., Wang,
860 J., Liu, Y., 2021. APICraft: Fuzz driver generation for closed-source sdk
861 libraries, in: 30th USENIX Security Symposium (USENIX Security 2021),
862 pp. 2811–2828.

863 Zhang, Q., Bai, X., Li, X., Duan, H., Li, Q., Li, Z., 2024a. Resolverfuzz:
864 Automated discovery of DNS resolver vulnerabilities with query-response
865 fuzzing, in: 33rd USENIX Security Symposium (USENIX Security 2024),
866 pp. 4729–4746.

867 Zhang, Q., Chu, Y., Shen, Y., Liu, J., Shi, H., Jiang, Y., Chang, W.,
868 2025. Tron: Fuzzing Linux network stack via protocol-system call pay-
869 load synthesis, in: 2025 40th IEEE/ACM International Conference on
870 Automated Software Engineering (ASE), pp. 3227–3238. doi:10.1109/
871 ase63991.2025.00266.

872 Zhang, Y., Fang, D., Liu, P., Xi, L., Lu, X., Chen, X., Si, S., Sun, L., 2024b.
873 MSGFuzzer: Message sequence guided industrial robot protocol fuzzing,
874 in: 2024 IEEE Conference on Software Testing, Verification and Validation
875 (ICST), pp. 140–150. doi:10.1109/icst60714.2024.00021.

876 Zhao, B., Li, Z., Qin, S., Ma, Z., Yuan, M., Zhu, W., Tian, Z., Zhang, C.,
877 2022. StateFuzz: System call-based state-aware linux driver fuzzing, in:
878 31st USENIX Security Symposium (USENIX Security 2022), pp. 3273–
879 3289.

880 Zhou, Z., Yang, Y., Wu, S., Huang, Y., Chen, B., Peng, X., 2024. Magneto:
881 A step-wise approach to exploit vulnerabilities in dependent libraries via

- 882 llm-empowered directed fuzzing, in: Proceedings of the 39th IEEE/ACM
883 International Conference on Automated Software Engineering, pp. 1633–
884 1644. doi:10.1145/3691620.3695531.
- 885 Zhu, X., Böhme, M., 2021. Regression greybox fuzzing, in: Proceedings of
886 the 2021 ACM SIGSAC Conference on Computer and Communications
887 Security, pp. 2169–2182. doi:10.1145/3460120.3484596.